Why zipping two dataloaders is problematic? And how to fix it with Semipy.

1 Intro

In this note, we study scenarios where it is necessary to simultaneously sample from multiple datasets or data sources. Such situations arise in various machine learning and statistical settings. A prominent example is deep semi-supervised learning (SSL), where, during training with stochastic gradient descent (SGD), each update step typically involves sampling one batch of labelled data and one batch of unlabelled data [Sohn et al., 2020, Xie et al., 2020, Schmutz et al., 2022]. These batches are then jointly used to compute the loss, evaluate gradients, and update the model's parameters. The need to synchronise the sampling process across datasets introduces subtle but important design choices in the data loading pipeline, which can significantly impact both learning dynamics and computational efficiency.

Another illustrative example is stratified sampling [Gopal, 2016], where samples must be drawn simultaneously from each stratum to compute an unbiased estimate of a global statistic. Similar ideas appear in the missing data literature, where the data are often partitioned according to missingness patterns. In such cases, one may need to draw samples independently within each pattern to estimate quantities such as expectations, imputations, or gradients in expectation–maximisation–type algorithms.

Most of the time, when confronted with this simultaneous sampling problem, practitioners tend to zip multiple dataloaders using standard PyTorch objects. Zipping is a Python operator that allow to iterate over a list of iterables (such as dataloaders) simultaneously. However, as we show in this note, this seemingly convenient solution is problematic. The zip operation stops as soon as one of the dataloaders is exhausted, triggering a reshuffling of all datasets before the next epoch begins. As a consequence, the datasets are not explored evenly, even after a large number of training iterations, which can introduce subtle biases in the sampling process. For instance, You et al. [2023] likely remarked this uneven exploration of the datasets as they created their own sampler to fix the uneven exploration [1].

In this note, we statistically analyse the implications of zipping dataloaders and then introduce a more principled solution, the JointSampler, implemented

[1] `https://github.com/charlesyou999648/ARCO/`

in our library SemiPy [2]. We first focus on the case of two dataloaders, as commonly encountered in semi-supervised learning, and later generalise our analysis to an arbitrary number of dataloaders. Through empirical experiments, we demonstrate that this issue can lead to performance degradation in deep semi-supervised learning when the training hyperparameters, such as batch size ratios, are not carefully tuned.

2 Simplified case: sampling from two dataloaders

2.1 Notations

Formally, we consider two datasets

$$\mathcal{D}_1 = (x_1, \ldots, x_{n_1}) \quad \text{and} \quad \mathcal{D}_2 = (\tilde{x}_1, \ldots, \tilde{x}_{n_2}),$$

drawn from distributions over domains $\mathcal{X}$ and $\tilde{\mathcal{X}}$, respectively. In many practical settings, such as semi-supervised learning under the missing at random assumption [Schmutz et al., 2022], these domains may coincide, $\mathcal{X} = \tilde{\mathcal{X}}$, but they may also differ in general. In SSL with different missing data mechanism, the domains may not coincide [Sportisse et al., 2023].

To build a dataloader from these datasets, to train a model for instance, the practitioner must specify a *batch size* for each: b_1 for $\mathcal{D}_1$ and b_2 for $\mathcal{D}_2$.

Each dataset is then iterated over in batches during processing. We define the *size* of a dataloader as the number of batches required to traverse the entire dataset once, which equals

$$|\mathcal{D}_1^{\text{loader}}| = \left\lceil \frac{n_1}{b_1} \right\rceil \quad \text{and} \quad |\mathcal{D}_2^{\text{loader}}| = \left\lceil \frac{n_2}{b_2} \right\rceil.$$

Without loss of generality, we assume

$$|\mathcal{D}_1^{\text{loader}}| \leq |\mathcal{D}_2^{\text{loader}}|,$$

meaning that dataset $\mathcal{D}_2$ requires more batches to be fully traversed than $\mathcal{D}_1$. This asymmetry commonly occurs when one dataset is significantly larger or when its batch size is smaller, as in semi-supervised learning where the unlabelled dataset is typically much larger than the labelled one [Sohn et al., 2020, Xie et al., 2020, Schmutz et al., 2022, You et al., 2023].

2.2 The problem with zipping dataloaders.

A common way to sample simultaneously from both datasets in PyTorch is to use the built-in `zip` operator as in the code snippet 1.

However, the length of this joint iterator is determined by the shortest dataloader:

$$|\texttt{zip}(\texttt{loader}_1, \texttt{loader}_2)| = \min\left(|\mathcal{D}_1^{\text{loader}}|, |\mathcal{D}_2^{\text{loader}}|\right) = |\mathcal{D}_1^{\text{loader}}|.$$

[2] `https://github.com/SemiPy/semipy.github.io/`

```python
from torch.utils.data import DataLoader
# Create dataloader
dataloader_1 = DataLoader(D1, batch_size=b1, shuffle=True)
dataloader_2 = DataLoader(D2, batch_size=b2, shuffle=True)
# Zip and sample.
for (x_1, x_2) in zip(dataloader_1,dataloader_2):
    ...
```

Listing 1: Building and zipping two dataloaders using Pytorch.

Consequently, the iteration stops as soon as the smaller dataloader is exhausted. After the iteration stopped, two situations may arise:

1. If `shuffle` is set to `True` in both dataloaders, they are reshuffled after each epoch, even though $\mathcal{D}_2$ has not been fully traversed. This mechanism results in an *uneven exploration* of the larger dataset $\mathcal{D}_2$, as some of its samples may never be seen before reshuffling.

2. If `shuffle` is set to `False` in both dataloaders, they are never reshuffled and part of $\mathcal{D}_2$ is never explored.

Consequently, a first good reflex is to always set `shuffle` to `True` and we consider `shuffle` set to `True` in the following for both dataloaders.

Over many epochs, the *uneven exploration* of the larger dataset introduces a systematic sampling bias that can negatively affect the learning dynamics in semi-supervised learning, for instance. In fact, during each pass through the zip of datasets, as $\mathcal{D}_2$ is shuffled, the probability of seeing a sample from $\mathcal{D}_2$ is given by $p = T \cdot \frac{b_2}{n_2}$, where $T = \lceil n_1/b_1 \rceil$ is the number of iterations per pass through the zip. On the other hand, the probability of seeing a sample from $\mathcal{D}_1$ is exactly 1.

Visibility Probability Therefore, we can study the visibility probability defined as the probability of seeing at least once a sample x from $\mathcal{D}_2$ during the process after a number of iterations $t \in \mathbb{N}^*$. We pose the Euclidean division of t by T as $t = qT + r$, with $q \in \mathbb{N}$ and $r \in [\![0, q-1]\!]$. To reach the t-th iterations, we pass through the zip q times, reshuffle each time, and then we sample r iterations through the zip. As each pass through is independent and identitically distributed, the probability of not seeing the sample x through a pass is $1 - p$. Then, in the remaining iterations, the probability of not seeing the sample x is $1 - \frac{rb_2}{n_2}$. Therefore, if we pose X, the iteration at which a specific sample $x \in \mathcal{D}_2$ is seen for the first time, we have:

$$P(X \leq t) = 1 - (1-p)^q \left(1 - \frac{rb_2}{n_2}\right), \quad p = T \cdot \frac{b_2}{n_2}. \tag{1}$$

We note that this probability converges to 1 with respect to t but is never equal to 1. However, we can compute the number of iterations $t^* = q^* + r^*$ needed to have a certain level of confidence $\alpha \in [0, 1[$ of seeing x:

To ensure $P(X \leq t) \geq \alpha$, we need:

$$q^* = \left\lfloor \frac{\ln(1-\alpha)}{\ln(1-p)} \right\rfloor, \tag{2}$$

$$r^* = \left\lceil \left(1 - \frac{1-\alpha}{(1-p)^q}\right) \frac{n_2}{b_2} \right\rceil. \tag{3}$$

Exemple: We take an example from Fixmatch paper [Sohn et al., 2020] using Cifar-10 with $n_1 = 40$, $n_2 = 49960$,$b_1 = 64$ and $b_2 = 7 * 64$. These are the hyperparameters for an experiment of the original Fixmatch's article [Sohn et al., 2020]. In this case, $\mathcal{D}_1$ is fully traversed after one iteration, while $\mathcal{D}_2$ have only been partially seen (0.89% coverage). In this situation, $T = 1$ and $P(X \leq t) \geq 0.95$ for $t = 333$ (i.e. 333 pass through the zip) and $P(X \leq t) \geq 0.99$ for $t = 512$ (i.e. 512 pass through the zip).

Expectation and Variance Additionally, we can study the number of times a sample x can be seen during K passes on the zip. We define, N_x as the number of times x is observed during the K passes. Seeing x during a pass is a Bernoulli law of probability p; therefore, N_x follows a binomial law of parameter (K, p). N_x takes its values in $[\![0, K]\!]$. We then have:

$$E[N_x] = Kp = KT \cdot \frac{b_2}{n_2}, \tag{4}$$

$$V[N_x] = Kp(1-p). \tag{5}$$

Exemple: With the same exemple as before, if $K = 333$, $E[N_x] \simeq 2.99$ and $V[N_x] \simeq 2.96$. If $K = 512$, $E[N_x] \simeq 4.59$ and $V[N_x] \simeq 4.55$. We observe a very high variance of N_x, which is not a desired behaviour, as we want control of what our model is seeing through a learning process.

Batch proportions are not kept Another major issue is that the batch proportion in the last batch of a pass through is not kept if b_1 does not divide n_1. For instance, if we train an SSL model using Fixmatch [Sohn et al., 2020] with 40 labelled samples and choose a labelled batch size of 39. Zipping these dataloaders will lead to two batches per pass through, one of 39 labelled samples and another one of 1 labelled sample. On the other hand, the unlabelled batch size is stable. So every even iterations, the learning process is using only one labelled sample, leading to a highly variable estimate of the gradient's risk, thus, instable learning process.

2.3 The solution using the JointSampler

To overcome this issue, we introduce a more principled solution, the JointSampler, implemented in our library SemiPy. The idea is to desynchronise the shuffling of $\mathcal{D}_1$ and $\mathcal{D}_2$. Indeed, when the dataset $\mathcal{D}_1$ is emptied before the dataset $\mathcal{D}_2$, only the dataset $\mathcal{D}_1$ is reshuffled for the next iteration; the dataset $\mathcal{D}_2$ continues from where it left off. Once the dataset $\mathcal{D}_2$ is emptied, it is reshuffled, and so on:

```python
from torch.utils.data import DataLoader
import semipy as smp
# Create dataloader
sampler = smp.sampler.JointSampler(dataset=[D1,D2],
    batch_size=10, label_prop=0.2)
dataloader = DataLoader([D1,D2], batch_sampler=sampler)
# Zip and sample.
for x in dataloader:
    ...
```

Listing 2: Building and zipping two dataloaders using Pytorch.

- $\mathcal{D}_1$ is reshuffled after exhaustion.
- $\mathcal{D}_2$ continues from where it left off until fully traversed.
- Then $\mathcal{D}_2$ is reshuffled.

Moreover, the JointSampler always keep the proportion fixed by the practitioner throughtout the process. This solution is straightforward to implement, as shown on code snipper 2, and tackle the issue highlighted for the zipping.

Visibility Probability In this scenario, the probability of seeing x is exactly 1 after on pass trhough the dataset $\mathcal{D}_2$, so $\lceil n_2/b_2 \rceil$ iterations. Therefore:

$$P(X \leq t) = 1\{t < \lceil n_2/b_2 \rceil\}\frac{tb_2}{n_2} + 1\{t \geq \lceil n_2/b_2 \rceil\}, \tag{6}$$

meaning that any sample x is always seen before the iterations $\lceil n_2/b_2 \rceil$.

Additionally, for any $t \in [0, \lceil n_2/b_2 \rceil]$, we prove that $P(X \leq t) \geq \alpha$ for any $\alpha \in [0, 1[$ implies:

$$t = \left\lceil \alpha \frac{n_2}{b_2} \right\rceil$$

.

Exemple: With the same exemple from Fixmatch [Sohn et al., 2020], we have, $P(X \leq t) \geq 0.95$ for $t = 106$ and $P(X \leq t) \geq 0.99$ for $t = 111$. We need much less iterations for the same level of confidence to have seen x. Moreover, after $\lceil n_2/b_2 \rceil = 112$, we are sure to have seen x.

Expectation and Variance With the JointSampler, any sample x from the dataset $\mathcal{D}_2$ is seen at least one time every $T_2 = \lceil n_2/b_2 \rceil$ iterations. If we pose $K_2 = \left\lfloor \frac{TK}{T_2} \right\rfloor$, the number of times we cycle on the dataset $\mathcal{D}_2$ for K cycle through $\mathcal{D}_1$, we have that N_x takes its values in $[\![K_2, K]\!]$. In the last batches of each pass through $\mathcal{D}_2$, if b_2 does not divide n_2, the batch is filled with $n_2 - \lfloor n_2/b_2 \rfloor b_2$ random samples. So, for every T_2 pass through $\mathcal{D}_2$, x is seen at least once and additionally once with probability $p_2 = \frac{n_2 - \lfloor n_2/b_2 \rfloor b_2}{n_2} = 1 - \lfloor n_2/b_2 \rfloor \frac{b_2}{n_2}$ (corresponding in being randomly sample in the last batch).

Finally, after the T_2 passes through, $\mathcal{D}_2$ is reshuffled and we sample $(TK - K_2T_2)$ batches to reach K pass through $\mathcal{D}_1$. So we prove that, the number of times x is observed during K epochs is exactly:

$$N_x = K_2 + \mathcal{B}(K_2, p_2) + \mathcal{B}\left((TK - K_2T_2)\frac{b_2}{n_2}\right)$$

.

Therefore, we have:

$$E[N_x] = K_2 + K_2p_2 + (TK - K_2T_2)\frac{b_2}{n_2}$$

,

$$V[N_x] = K_2p_2(1-p_2) + (TK - K_2T_2)\frac{b_2}{n_2}(1 - (TK - K_2T_2)\frac{b_2}{n_2})$$

.

Exemple: With the same exemple as before, if $K = 333$, we have $T_2 = 112$ and therefore, x is seen at least $K_2 = 2$ times. Then, $E[N_x] \simeq 3.01$ and $V[N_x] \simeq 0.01$. If $K = 512$, we have $T_2 = 112$ and therefore, x is seen at least $K_2 = 4$ times. Then, $E[N_x] \simeq 4.63$ and $V[N_x] \simeq 0.26$.

Problems fixed: To sum up, the JointSampler theoretically fixe the two issues highlighted for the zipping situation; JointSampler

- Does keep the proportion in the last batch if b_1 does not divide n_1.
- Does balance on the sampling in $\mathcal{D}_2$.

3 What if we have more than two datasets?

The situation naturally extends to the case where we have access to $M > 2$ datasets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_M$, each associated with its own batch size $b_1, b_2, \ldots, b_M$. As before, we define the number of batches required to traverse each dataset once as

$$T_i = \left\lceil \frac{n_i}{b_i} \right\rceil, \quad i \in [\![1, M]\!].$$

Without loss of generality, we assume $T_1 \leq T_2 \leq \cdots \leq T_M$, so that $\mathcal{D}_1$ is the *smallest* dataloader, i.e., the one exhausted first.

Zipping more than two dataloaders. When using the standard `zip` operation on multiple dataloaders in PyTorch, the situation only worsens. The iteration length of the joint process becomes $|\texttt{zip(loader}_1, \ldots, \texttt{loader}_M)| = \min_i T_i = T_1$. Therefore, the joint iterator stops as soon as the smallest dataloader, $\mathcal{D}_1$, is exhausted. All dataloaders are then reshuffled simultaneously before the next epoch. As a result, none of the larger datasets $\mathcal{D}_2, \ldots, \mathcal{D}_M$ are fully traversed before reshuffling, leading to a severe imbalance in visibility probabilities and an uneven exploration of the data.

Generalisation of the JointSampler. The `JointSampler` naturally generalises to any number of datasets. The idea remains the same: each dataset is reshuffled *independently* upon exhaustion, while the others continue from where they left off. This asynchronous mechanism ensures that:

- each dataset $\mathcal{D}_i$ is fully traversed before reshuffling, preserving full coverage;
- the user-defined batch-size ratios between datasets remain constant throughout training;
- the overall iteration process is stable and statistically balanced, independently of the relative sizes of the datasets.

Formally, the visibility probability for any sample $x \in \mathcal{D}_i$ converges to 1 within T_i iterations ensuring that all datasets are evenly explored according to their own temporal scales. Moreover, its behaviour remains consistent regardless of the number or relative sizes of datasets, whereas `zip`-based strategies become increasingly inefficient and biased as M grows.

4 Experiments

4.1 Simple comparison

In this experiment, we compute the simple statistics we analysed earlier for the Fixmatch example using Cifar-10 with $n_1 = 40$, $n_2 = 49960$,$b_1 = 64$ and $b_2 = 7 * 64$. In particular, we compare on Figure 1 the probability of event X given iteration t, the number of iterations needed to reach various levels of confidence α for the probability of seeing x and the behaviour of the expectation and variance of N_x with respect to K, the number of pass on $\mathcal{D}_1$.

4.2 Risk of performance degradation during training

To empirically show the risk of sampling from two dataloaders while zipping and the benefit of using the JointSampler that conserve the batch ratio, we train a SSL model using Fixmatch [Sohn et al., 2020] with 40 labelled samples. We volontarly choice a hard case for the zipping where the practitioner chose to use a labelled batch size of 39. Zipping these dataloaders will lead to two batches per epoch, one of 39 samples and another one of 1. On the other hand, the Jointsampler will mantain the batch size ratio between labelled and unlabelled throughout the training.

5 Discussion

The *JointSampler* significantly improves fairness and regularity in sample access from large datasets. It ensures proportionality in final batches and reduces variance compared to zipped dataloaders.

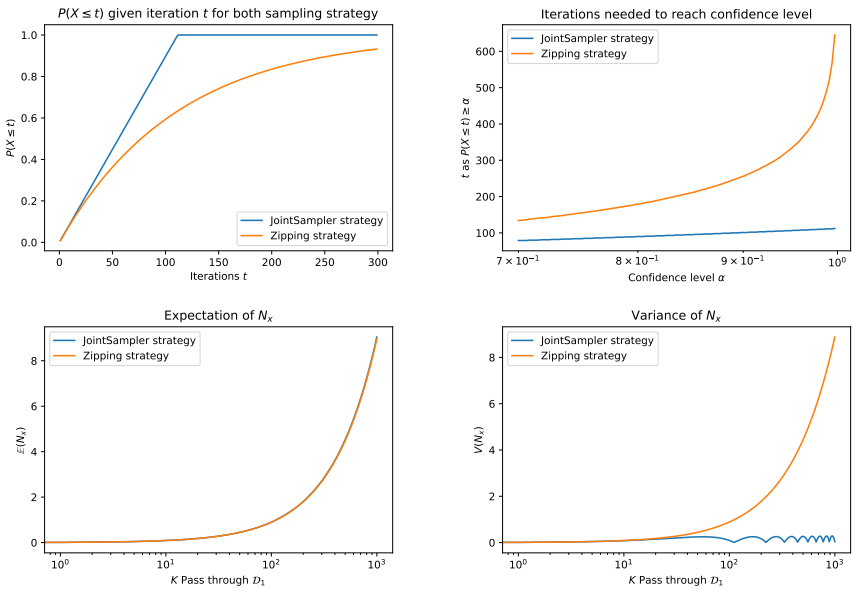

Figure 1: Stats.

6 Conclusion

We demonstrated that zipping dataloaders introduces sampling biases that can negatively affect SSL. Our proposed *JointSampler* offers a principled and effective solution. We advocate for its adoption in SSL implementations. **In the doubt, use JointSampler rather than zipping.**

Note - TODO Commenter la solution de lighthning: CombinedLoader

Fixmatch: 40 labelled batch size 39 et regarder la différence entre les trois méthodos. Suivre loss et résultats. faire avec Semipy. Quel est le nombre de points qui a été vu à T. (pour l'axe des x) plutôt que le nombre d'itérations.

exp: Si SSL marche => Stratifié `https://arxiv.org/pdf/2006.05988` `https://proceedings.mlr.press/v48/gopal16.pdf` `https://ieeexplore.ieee.org/abstract/document/9078901`

References

S. Gopal. Adaptive sampling for sgd by exploiting side information. In *International Conference on Machine Learning*, pages 364–372. PMLR, 2016.

H. Schmutz, O. Humbert, and P.-A. Mattei. Don't fear the unlabelled: safe semi-supervised learning via debiasing. In *The Eleventh International Conference on Learning Representations*, 2022.

K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *Advances in neural information processing systems*, 33:596–608, 2020.

A. Sportisse, H. Schmutz, O. Humbert, C. Bouveyron, and P.-A. Mattei. Are labels informative in semi-supervised learning? Estimating and leveraging the missing-data mechanism. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32521–32539. PMLR, 23–29 Jul 2023. URL `https://proceedings.mlr.press/v202/sportisse23a.html`.

Q. Xie, Z. Dai, E. Hovy, T. Luong, and Q. Le. Unsupervised data augmentation for consistency training. *Advances in neural information processing systems*, 33:6256–6268, 2020.

C. You, W. Dai, Y. Min, F. Liu, D. Clifton, S. K. Zhou, L. Staib, and J. Duncan. Rethinking semi-supervised medical image segmentation: A variance-reduction perspective. *Advances in neural information processing systems*, 36:9984–10021, 2023.